

Week 1: thresholded Branin active-learning loop

Branin is the measurable Week 1 test. Its threshold boundary is known, so after every labelled evaluation we can compute the exact fraction of test points classified incorrectly.

```
In [2]: from pathlib import Path
import json
import sys

ROOT = Path.cwd().resolve()
if ROOT.name == 'notebooks':
    ROOT = ROOT.parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from IPython.display import Image, display
from src.branin_week1 import run_experiment
```

Run three reproducible seeds

Each seed gets a 1,500-point pool and an independent 4,000-point test set. Only the selected pool label is revealed at each iteration.

```
In [3]: output_dir = ROOT / 'outputs' / 'branin_week1'
runs = run_experiment(output_dir=output_dir)
summary = json.loads((output_dir / 'summary.json').read_text(encoding='utf-8'))
print(f"Threshold: {summary['threshold']:.6f}")
print(f"Mean error: {summary['mean_initial_error']:.3f} -> {summary['mean_final_error']:.3f}")
```

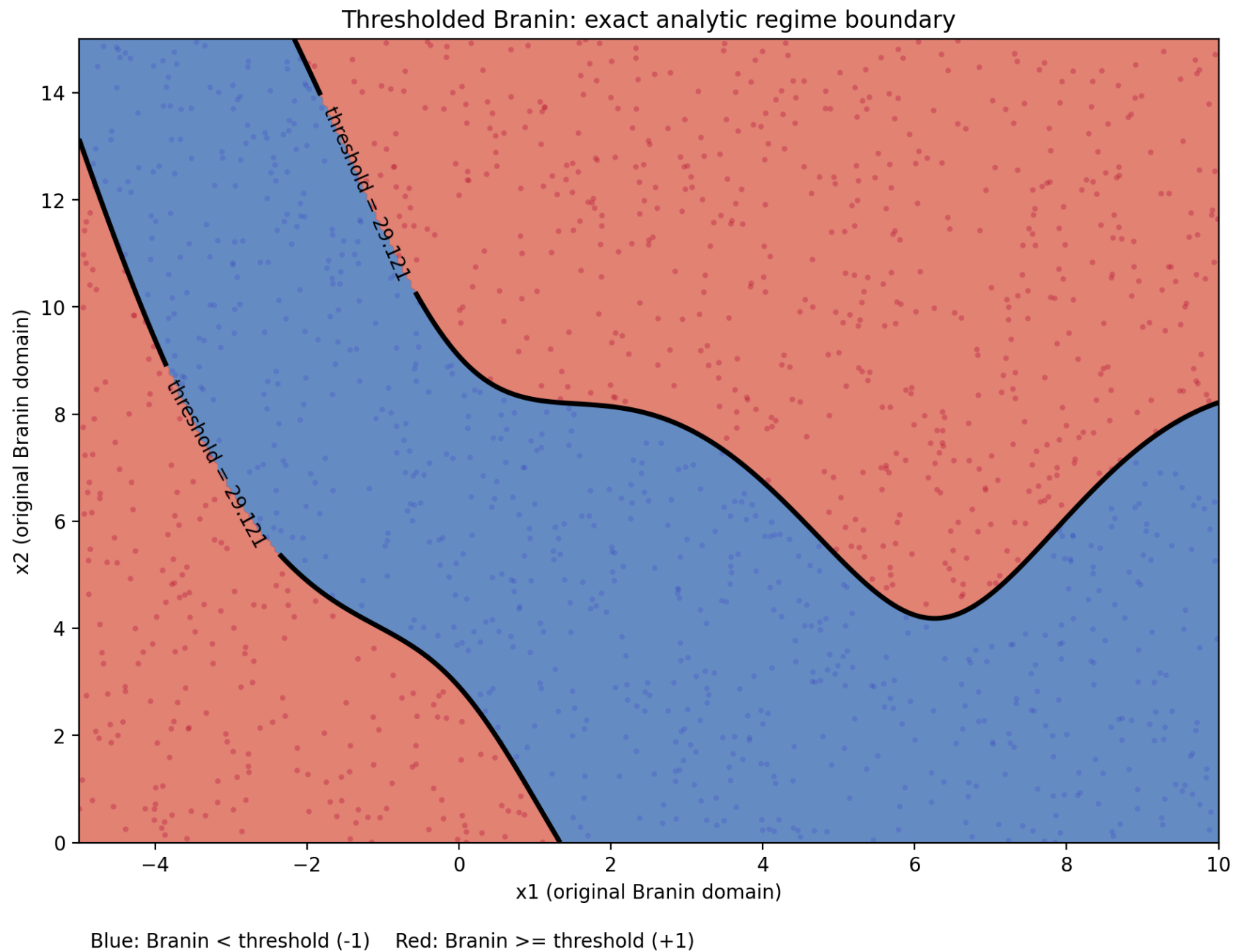
Threshold: 29.121261

Mean error: 0.337 -> 0.060

A. Exact thresholded-Branin dataset

The black curve is the exact boundary, not a model prediction.

```
In [4]: display(Image(filename=str(output_dir / 'dataset_overview.png')))
```

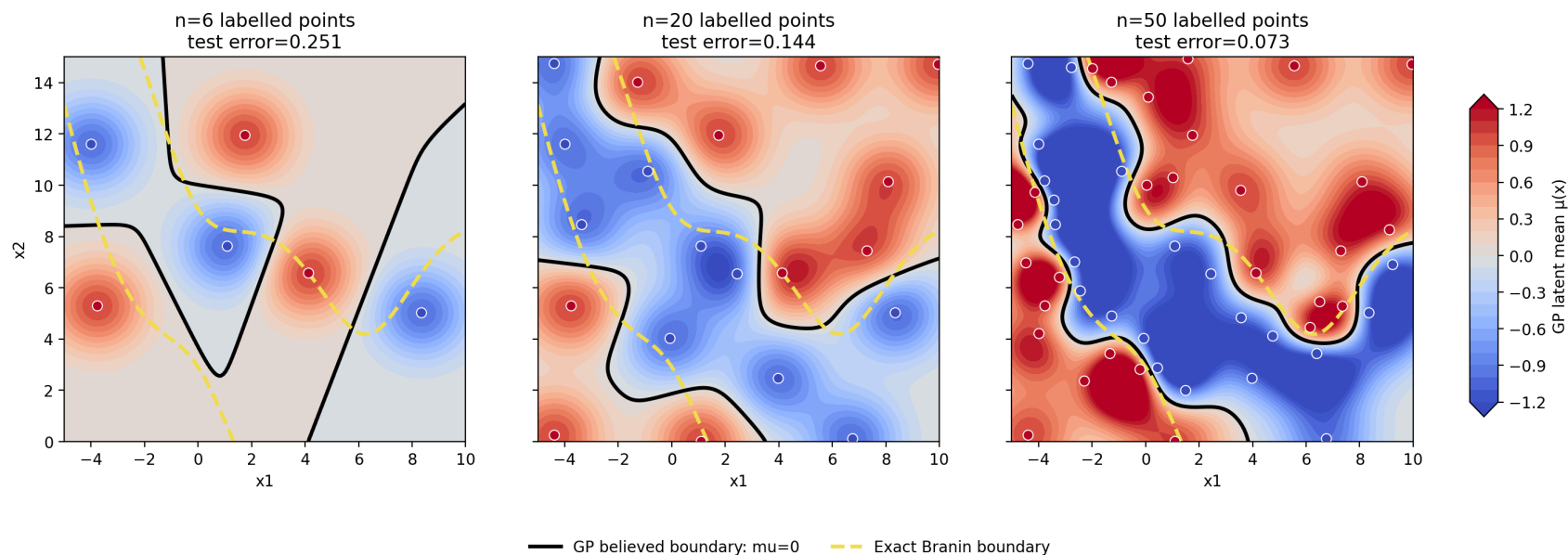


B. Boundary learning at 6, 20, and 50 labels

Black is the GP's current $\mu=0$ boundary; yellow dashed is the exact Branin boundary.

```
In [5]: display(Image(filename=str(output_dir / 'active_learning_snapshots_seed0.png')))
```

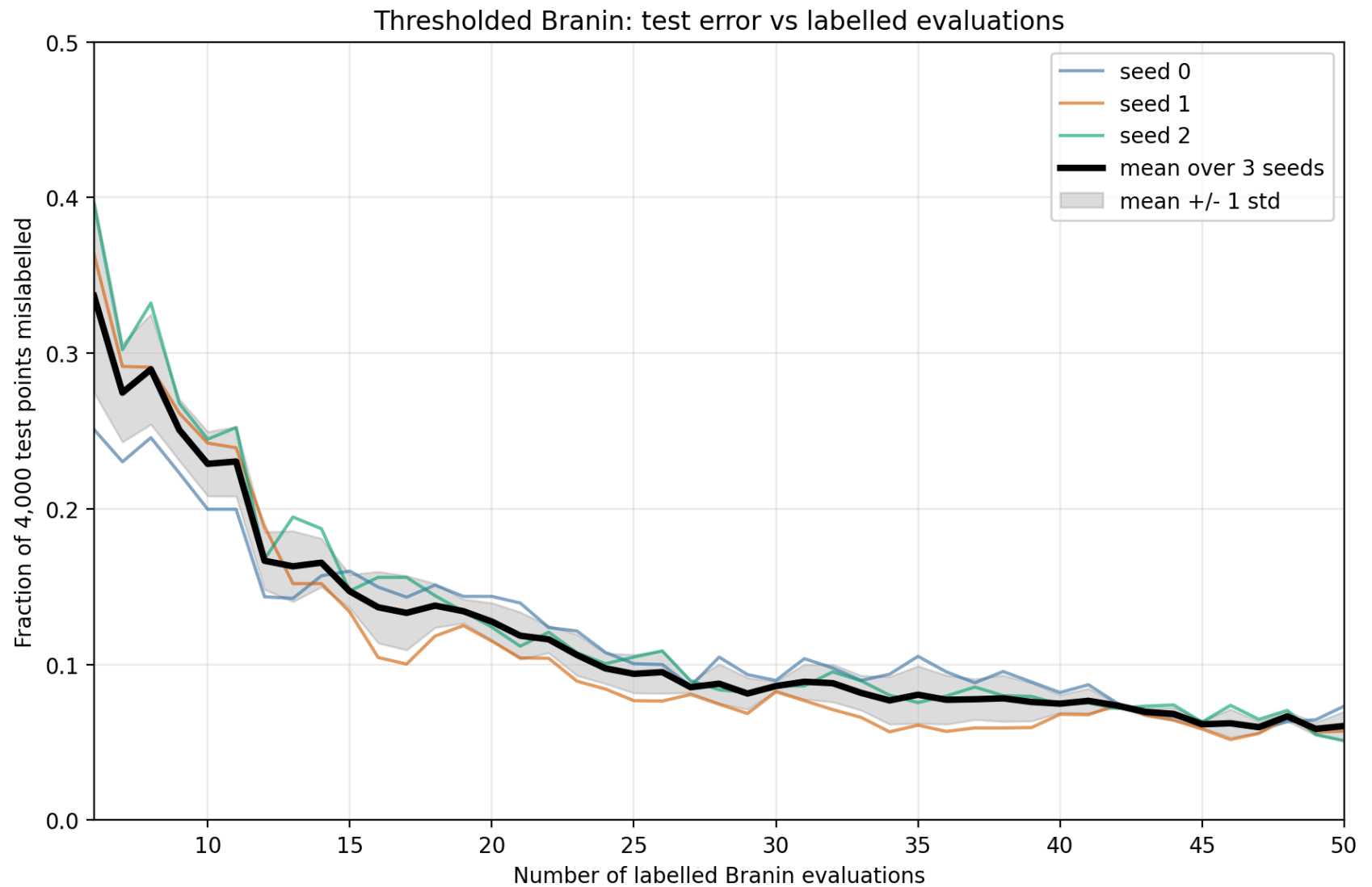
Week 1 active learning on thresholded Branin (seed 0)



C. Numeric error against the exact labels

The curve may move upward at individual steps. The important check is the overall trend over the full budget and across seeds.

```
In [6]: display(Image(filename=str(output_dir / 'error_vs_evaluations.png')))  
{seed: {'initial': values['initial_error'], 'final': values['final_error']} for seed, values in summary['per_seed'].}
```



```
Out[6]: {'0': {'initial': 0.251, 'final': 0.07325},  
        '1': {'initial': 0.3635, 'final': 0.05725},  
        '2': {'initial': 0.396, 'final': 0.051}}
```

```
In [7]: import json  
import pandas as pd  
  
with open("../outputs/branin_week1/summary.json", "r") as f:
```

```
summary = json.load(f)

summary.keys()
```

```
Out[7]: dict_keys(['experiment', 'threshold', 'threshold_percentile', 'threshold_estimation_seed', 'threshold_estimation_sample_size', 'domain', 'pool_size_per_seed', 'test_size_per_seed', 'initial_labelled_size', 'total_budget', 'seeds', 'selection_rule', 'prediction_rule', 'error_definition', 'initial_design', 'model_note', 'kernel_note', 'per_seed', 'mean_initial_error', 'mean_final_error', 'all_seeds_finish_below_initial_error'])
```

```
In [8]: rows = []

for seed, result in summary["per_seed"].items():
    rows.append({
        "seed": seed,
        "initial_error": result["initial_error"],
        "final_error": result["final_error"],
        "improved": result["final_error_lower_than_initial"],
        "initial_design_attempts": result["initial_design_resampling_attempts"],
    })

df = pd.DataFrame(rows)
df
```

```
Out[8]:
```

	seed	initial_error	final_error	improved	initial_design_attempts
0	0	0.2510	0.07325	True	1
1	1	0.3635	0.05725	True	1
2	2	0.3960	0.05100	True	2

```
In [9]: general_info = {
    "experiment": summary["experiment"],
    "threshold": summary["threshold"],
    "threshold_percentile": summary["threshold_percentile"],
    "threshold_estimation_seed": summary["threshold_estimation_seed"],
    "threshold_estimation_sample_size": summary["threshold_estimation_sample_size"],
    "x1_domain": summary["domain"]["x1"],
    "x2_domain": summary["domain"]["x2"],
    "pool_size_per_seed": summary["pool_size_per_seed"],
    "test_size_per_seed": summary["test_size_per_seed"],
```

```
"initial_labelled_size": summary["initial_labelled_size"],
"total_budget": summary["total_budget"],
"seeds": summary["seeds"],
"selection_rule": summary["selection_rule"],
"prediction_rule": summary["prediction_rule"],
"error_definition": summary["error_definition"],
"initial_design": summary["initial_design"],
"model_note": summary["model_note"],
}

general_df = pd.DataFrame(
    [{"field": key, "value": value} for key, value in general_info.items()]
)

general_df
```

Out[9]:

	field	value
0	experiment	thresholded_branin_week1
1	threshold	29.121261
2	threshold_percentile	45.0
3	threshold_estimation_seed	2026
4	threshold_estimation_sample_size	20000
5	x1_domain	[-5.0, 10.0]
6	x2_domain	[0.0, 15.0]
7	pool_size_per_seed	1500
8	test_size_per_seed	4000
9	initial_labelled_size	6
10	total_budget	50
11	seeds	[0, 1, 2]
12	selection_rule	choose unlabelled pool point with smallest abs...
13	prediction_rule	+1 if GP latent mean $\mu \geq 0$, else -1
14	error_definition	fraction of exact test labels predicted incorr...
15	initial_design	Six points are sampled uniformly without repla...
16	model_note	GaussianProcessRegressor on {-1,+1} labels is ...

In [10]: seed_rows = []

```

for seed, result in summary["per_seed"].items():
    seed_rows.append({
        "seed": int(seed),
        "initial_error": result["initial_error"],
        "final_error": result["final_error"],
        "absolute_improvement": result["initial_error"] - result["final_error"],
    })

```

```

        "relative_improvement_%": 100 * (result["initial_error"] - result["final_error"]) / result["initial_error"],
        "final_lower_than_initial": result["final_error_lower_than_initial"],
        "initial_design_attempts": result["initial_design_resampling_attempts"],
    })

seed_df = pd.DataFrame(seed_rows)

seed_df

```

Out[10]:

	seed	initial_error	final_error	absolute_improvement	relative_improvement_%	final_lower_than_initial	initial_design_attempts
0	0	0.2510	0.07325	0.17775	70.816733	True	1
1	1	0.3635	0.05725	0.30625	84.250344	True	1
2	2	0.3960	0.05100	0.34500	87.121212	True	2

In [11]:

```

seed_df_style = seed_df.style.format({
    "initial_error": "{:.2%}",
    "final_error": "{:.2%}",
    "absolute_improvement": "{:.2%}",
    "relative_improvement_%": "{:.1f}%",
})

seed_df_style

```

Out[11]:

	seed	initial_error	final_error	absolute_improvement	relative_improvement_%	final_lower_than_initial	initial_design_attempts
0	0	25.10%	7.32%	17.78%	70.8%	True	1
1	1	36.35%	5.73%	30.62%	84.3%	True	1
2	2	39.60%	5.10%	34.50%	87.1%	True	2

In [12]:

```

mean_summary_df = pd.DataFrame([
    "mean_initial_error": summary["mean_initial_error"],
    "mean_final_error": summary["mean_final_error"],
    "mean_absolute_improvement": summary["mean_initial_error"] - summary["mean_final_error"],
    "mean_relative_improvement_%": 100 * (summary["mean_initial_error"] - summary["mean_final_error"]) / summary["me

```

```

    "all_seeds_finish_below_initial_error": summary["all_seeds_finish_below_initial_error"],
  })

mean_summary_df.style.format({
    "mean_initial_error": "{:.2%}",
    "mean_final_error": "{:.2%}",
    "mean_absolute_improvement": "{:.2%}",
    "mean_relative_improvement_%": "{:.1f}%",
  })

```

Out[12]:

	mean_initial_error	mean_final_error	mean_absolute_improvement	mean_relative_improvement_%	all_seeds_finish_below_initial_
0	33.68%	6.05%	27.63%	82.0%	



In [13]:

```

history_rows = []

for seed, result in summary["per_seed"].items():
    budgets = result["budgets"]
    errors = result["error_history"]

    for budget, error in zip(budgets, errors):
        history_rows.append({
            "seed": int(seed),
            "budget": budget,
            "error": error,
        })

history_df = pd.DataFrame(history_rows)

history_df.head()

```

```
Out[13]:
```

	seed	budget	error
0	0	6	0.25100
1	0	7	0.23025
2	0	8	0.24575
3	0	9	0.22300
4	0	10	0.19975

```
In [14]: history_df
```

```
Out[14]:
```

	seed	budget	error
0	0	6	0.25100
1	0	7	0.23025
2	0	8	0.24575
3	0	9	0.22300
4	0	10	0.19975
...	...	...	...
130	2	46	0.07375
131	2	47	0.06475
132	2	48	0.07050
133	2	49	0.05500
134	2	50	0.05100

135 rows × 3 columns

```
In [15]: error_wide_df = history_df.pivot(  
        index="budget",  
        columns="seed",
```

```
        values="error"  
    )  
  
    error_wide_df["mean_error"] = error_wide_df.mean(axis=1)  
    error_wide_df["std_error"] = error_wide_df[[0, 1, 2]].std(axis=1)  
  
    error_wide_df.head()
```

Out[15]:

	seed	0	1	2	mean_error	std_error
budget						
6	0.25100	0.36350	0.39600	0.336833	0.076089	
7	0.23025	0.29150	0.30225	0.274667	0.038840	
8	0.24575	0.29100	0.33225	0.289667	0.043265	
9	0.22300	0.26150	0.26775	0.250750	0.024235	
10	0.19975	0.24225	0.24475	0.228917	0.025290	

In [16]: error_wide_df

Out[16]:

	seed	0	1	2	mean_error	std_error
budget						
6	0.25100	0.36350	0.39600	0.336833	0.076089	
7	0.23025	0.29150	0.30225	0.274667	0.038840	
8	0.24575	0.29100	0.33225	0.289667	0.043265	
9	0.22300	0.26150	0.26775	0.250750	0.024235	
10	0.19975	0.24225	0.24475	0.228917	0.025290	
11	0.19975	0.23925	0.25225	0.230417	0.027342	
12	0.14350	0.18875	0.16775	0.166667	0.022644	
13	0.14250	0.15200	0.19475	0.163083	0.027832	
14	0.15700	0.15200	0.18725	0.165417	0.019073	
15	0.16000	0.13375	0.14725	0.147000	0.013127	
16	0.14975	0.10450	0.15600	0.136750	0.028104	
17	0.14325	0.10025	0.15600	0.133167	0.029211	
18	0.15100	0.11825	0.14425	0.137833	0.017292	
19	0.14375	0.12500	0.13400	0.134250	0.009377	
20	0.14375	0.11500	0.12400	0.127583	0.014706	
21	0.13950	0.10425	0.11175	0.118500	0.018569	
22	0.12375	0.10400	0.12075	0.116167	0.010643	
23	0.12150	0.08925	0.10750	0.106083	0.016172	
24	0.10775	0.08425	0.10050	0.097500	0.012034	
25	0.10050	0.07675	0.10475	0.094000	0.015089	
26	0.10000	0.07650	0.10875	0.095083	0.016678	

seed	0	1	2	mean_error	std_error
budget					
27	0.08600	0.08075	0.08950	0.085417	0.004404
28	0.10475	0.07450	0.08375	0.087667	0.015501
29	0.09350	0.06850	0.08200	0.081333	0.012513
30	0.08975	0.08250	0.08625	0.086167	0.003626
31	0.10375	0.07675	0.08625	0.088917	0.013696
32	0.09775	0.07100	0.09525	0.088000	0.014775
33	0.08975	0.06600	0.08950	0.081750	0.013640
34	0.09375	0.05675	0.08025	0.076917	0.018724
35	0.10525	0.06100	0.07550	0.080583	0.022559
36	0.09525	0.05700	0.07975	0.077333	0.019239
37	0.08825	0.05925	0.08550	0.077667	0.016008
38	0.09550	0.05925	0.08000	0.078250	0.018188
39	0.08875	0.05950	0.07950	0.075917	0.014951
40	0.08200	0.06800	0.07450	0.074833	0.007006
41	0.08700	0.06775	0.07525	0.076667	0.009703
42	0.07525	0.07400	0.07175	0.073667	0.001774
43	0.06800	0.06750	0.07325	0.069583	0.003185
44	0.06650	0.06425	0.07400	0.068250	0.005105
45	0.06325	0.05850	0.06300	0.061583	0.002673
46	0.06125	0.05175	0.07375	0.062250	0.011034
47	0.05875	0.05575	0.06475	0.059750	0.004583

seed	0	1	2	mean_error	std_error
budget					
48	0.06325	0.06625	0.07050	0.066667	0.003643
49	0.06450	0.05650	0.05500	0.058667	0.005107
50	0.07325	0.05725	0.05100	0.060500	0.011476

In []: